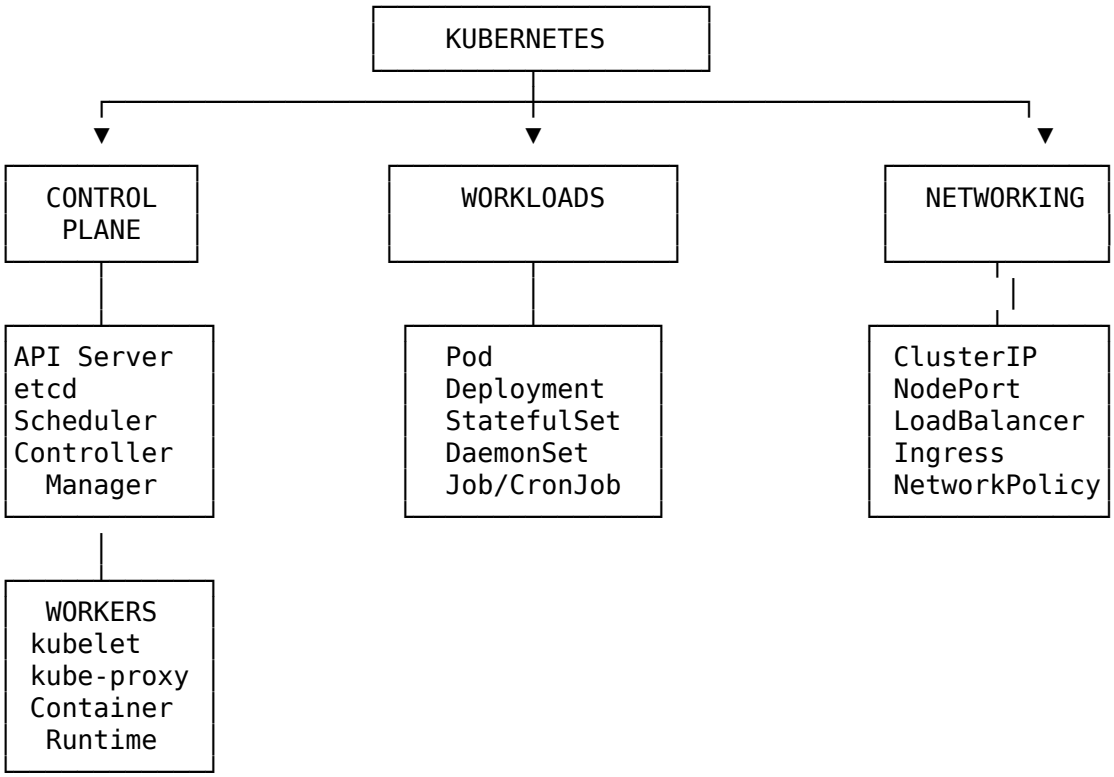
Contents

Kubernetes — Brain Map **1**
Full Concept Tree 1
Request Flow Visual 3
Pod Lifecycle State Machine 3
Debug Decision Tree 3
kubectl Muscle Memory 3

Kubernetes — Brain Map



Full Concept Tree

```
KUBERNETES
├── ARCHITECTURE
│   ├── Control Plane
│   │   ├── API Server — REST gateway, auth/authz, validates, stores to etcd
│   │   ├── etcd — distributed KV store, source of truth, Raft consensus
│   │   ├── Scheduler — watches unscheduled pods, picks node (predicates→priorities)
│   │   └── Controller Manager — runs control loops (Deployment, Node, Endpoint...)
│   └── Data Plane (Worker Nodes)
│       ├── kubelet — pod lifecycle, reports status, runs probes
│       ├── kube-proxy — iptables/ipvs rules for Service routing
│       └── Container Runtime — containerd/CRI-O (runs OCI containers)
│
├── WORKLOADS
│   ├── Pod — smallest unit, 1+ containers, shared net/storage
│   ├── Deployment — stateless, rolling updates, ReplicaSet management
│   ├── StatefulSet — ordered, sticky identity, stable storage
│   ├── DaemonSet — one pod per node (monitoring, logging, networking)
│   ├── Job — run to completion (batch)
│   └── CronJob — scheduled Jobs
│
```

```
├── CONFIGURATION & SECRETS
│   ├── ConfigMap — non-sensitive config (env vars, mounted files)
│   └── Secret — base64 encoded (not encrypted!) → use SOPS/Vault/ESO
│
├── NETWORKING
│   ├── Services
│   │   ├── ClusterIP — internal only, stable VIP
│   │   ├── NodePort — expose on node IP + port (30000-32767)
│   │   ├── LoadBalancer — cloud LB provisioned
│   │   └── ExternalName — CNAME alias
│   ├── Ingress — L7 routing (host/path → service)
│   ├── IngressClass — selects which controller handles ingress
│   ├── NetworkPolicy — L3/L4 firewall rules (requires CNI support)
│   └── CNI Plugins — Calico, Cilium, Flannel, Weave
│
├── STORAGE
│   ├── PersistentVolume (PV) — cluster resource (admin provisions)
│   ├── PersistentVolumeClaim (PVC) — user request for storage
│   ├── StorageClass — dynamic provisioning recipe
│   ├── VolumeMode: Filesystem or Block
│   └── AccessModes: RWO, ROX, RWX, RWOP
│
├── SECURITY
│   ├── RBAC — Role + ClusterRole + Binding
│   ├── ServiceAccount — pod identity for API access
│   ├── Pod Security Admission — privileged/baseline/restricted
│   ├── SecurityContext — user, capabilities, readOnlyRootFilesystem
│   ├── NetworkPolicy — pod-level firewall
│   └── IRSA (AWS) — pod ↔ IAM role via OIDC
│
├── SCHEDULING & PLACEMENT
│   ├── Requests & Limits — CPU/memory per container
│   ├── QoS classes — Guaranteed / Burstable / BestEffort
│   ├── NodeSelector — simple key=value node matching
│   ├── Affinity/Anti-Affinity — soft/hard rules
│   ├── Taints & Tolerations — repel pods, unless tolerated
│   └── TopologySpreadConstraints — balance across zones
│
├── AUTOSCALING
│   ├── HPA — scale pods based on CPU/memory/custom metrics
│   ├── VPA — right-size pod resource requests
│   ├── KEDA — event-driven autoscaling (Kafka, SQS, Prometheus)
│   └── Cluster Autoscaler — add/remove nodes based on pending pods
│
├── OBSERVABILITY
│   ├── kubectl top — resource usage (needs metrics-server)
│   ├── kubectl logs — container stdout/stderr
│   ├── kubectl describe — events, conditions, details
│   ├── kubectl exec — shell access for debugging
│   └── kubectl port-forward — local access to pod/service
│
└── ECOSYSTEM
    ├── Helm — package manager, templating, release management
    ├── ArgoCD/Flux — GitOps continuous delivery
    ├── Kyverno/OPA — admission control policies
    ├── cert-manager — TLS certificate automation
    ├── External Secrets Operator — secrets from Vault/AWS SM
    └── Operators — extend K8s with custom resources (CRD + controller)
```

Request Flow Visual

```
User → DNS → Cloud LB → NGINX Ingress Pod (Ingress rule matches)
                                ↓
                          Service (ClusterIP)
                                ↓
                        iptables / ipvs (kube-proxy)
                                ↓
                        Pod selected by Endpoint
                                ↓
                        Container (app process)
                                ↓
                        (→ external: via NAT Gateway)
```

Pod Lifecycle State Machine

```
Pending → ContainerCreating → Running → Succeeded
                ↓                ↓
          (image pull)    → Failed (OOMKilled, CrashLoopBackOff)
                              ↓
                        Terminating (preStop → SIGTERM → 30s → SIGKILL)
```

Debug Decision Tree

```
Pod not running?
    → kubectl get pod / kubectl describe pod
    → Status = Pending?
        → No nodes match? (NodeSelector/Taint/Resource?)
        → Image pull failing? (registry, credentials)
    → Status = CrashLoopBackOff?
        → kubectl logs --previous
        → Exit code 1 = app crash, 137 = OOMKilled, 139 = segfault
    → Status = Running but not serving?
        → kubectl exec → test internally
        → Readiness probe failing?
        → Service selector matching?

Service not reachable?
    → kubectl get endpoints svc-name
        → Empty? → selector doesn't match pod labels
        → Has IPs? → kube-proxy issue or NetworkPolicy blocking
```

kubectl Muscle Memory

```
# CONTEXT
kubectl config get-contexts           # list clusters
kubectl config use-context NAME       # switch cluster

# GET + FILTER
kubectl get pods -A -o wide           # all namespaces, node info
kubectl get pods --field-selector=status.phase=Running
kubectl get events --sort-by='.lastTimestamp'

# TROUBLESHOOT
kubectl describe pod NAME             # events + status
kubectl logs NAME -c CONTAINER --previous   # crash logs
kubectl exec -it NAME -- sh           # shell into pod
kubectl port-forward svc/NAME 8080:80 # local access

# APPLY + MANAGE
```

```
kubectl apply -f manifest.yaml        # declarative apply
kubectl rollout status deploy/NAME    # watch rollout
kubectl rollout undo deploy/NAME      # rollback one version
kubectl scale deploy NAME --replicas=5

# QUICK
kubectl run debug --image=busybox:latest --rm -it -- sh  # debug pod
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa
kubectl api-resources                 # all resource types
```